

EKF_GPS_Project -- Conversation transcript

Session date: 2026-08-10

Anchor: starts right after the git push of commit e23c304 (15:29:25), covering everything through the end of the session. Note: exact per-message timestamps aren't available to me, so the first exchange or two below may land slightly before 5pm; everything from the lambda_k/ANIS deep-dive onward is well after it.

[ASSISTANT]

Pushed successfully -- commit e23c304, 93 files, on origin/main.
Working tree is clean.

[USER]

why does vk have to be mean?

[ASSISTANT]

Because V_k is trying to estimate a statistical property -- the true, per-step innovation covariance -- from a handful of real samples, and covariance is defined as an average squared deviation, not a sum. Using the sum instead of the mean would break the comparison it's built for.

Covariance is an expected value by definition: $E[e.e^T]$. Since only $LAMBDA_WINDOW=10$ real samples are available, the standard way to estimate an expected value from a finite sample is to average them (the maximum-likelihood estimator for the covariance of a zero-mean Gaussian).

Why not just sum them? A sum scales with how many samples you happen to include -- summing 10 samples of $e.e^T$ would be roughly 10x too large compared to a single typical innovation, and doubling $LAMBDA_WINDOW$ would double the sum again with nothing in the actual data changing.

This matters specifically because of what V_k gets compared against: $M_k = H.(F.P_prior.F^T).H^T$ is a single-step theoretical quantity. For ratio = trace_N/trace_M to mean anything, V_k has to be on that same single-step footing. The mean gives you that; the sum doesn't. Averaging is what keeps lambda_k's scale independent of window size.

[USER]

explain then what is vk-r

[ASSISTANT]

$N_k = V_k - R$ strips out the one piece of the real, observed spread that's known and external -- GPS's own measurement noise -- leaving behind only the portion of the disagreement that's actually about the state/model.

V_k is the empirical estimate of the FULL innovation spread. In a well-calibrated filter, that full spread theoretically breaks into three pieces:

$$S = \underset{\wedge}{M_k} + \underset{\wedge}{H.Q.H^T} + \underset{\wedge}{R}$$

carried-	this-step's	GPS's own
over	accel	measurement
state	noise	noise
uncertainty		

V_k (built from real residuals) reflects all three. But M_k only captures the first piece, by design -- it excludes both Q and R.

So the subtraction is about making the comparison fair. If you skipped -R and compared raw V_k to M_k, you'd be comparing "everything, including GPS's fixed noise floor" against "only the state-carryover piece" -- and since R (25-100 for this GPS setup) is typically much bigger than M_k's steady-state range (single digits to low tens), that comparison would come out structurally inflated almost every step, attack or not.

Subtracting R first answers a cleaner question: "of everything actually happening in the residuals right now, how much can't be explained by the GPS receiver's own ordinary noise?" That's the piece diagnostic of a model/state problem -- what needs comparing against M_k.

[USER]

my idea was till the covariance is stable the ekf.p value will keep on getting inflated, that should show up in some counter right??? why does it not show up. I AM FRUSTRATED

[ASSISTANT]

[Reassured the user this was a sound, productive question, not a sign of missing something obvious.]

Your model: disagreement persists -> P keeps inflating -> should be an easy, growing signal. That's sound IF Kalman filtering let uncertainty accumulate. It doesn't:

Every single step, the update forcibly resets P, regardless of how big lambda_k just made it. This is the "collapse" mechanism: $1/P_{\text{post}} = 1/P_{\text{pred}} + 1/R$ (precision adds). No matter how much lambda_k inflates P_pred this step, P_post is bounded -- it approaches R's scale from below and can never exceed it. This happens fresh, every step, from scratch -- there's no accumulator carrying forward "how inflated has this been recently." Each step is: inflate briefly (pre-update) -> get pulled back down toward R (post-update) -> repeat.

So what should actually happen during a sustained attack isn't "P keeps growing" -- it's "P settles into a new, moderately elevated but

bounded steady state." If disagreement keeps recurring every step, λ_k keeps getting recomputed above 1 every step too, so P_{prior} stays a bit higher than the pre-attack baseline, consistently, for as long as the attack lasts -- but it's a shifted plateau, not a ramp.

This is actually consistent with what was seen in the `seed1_applied_stf hpmcounter3` plot earlier -- drift sitting measurably below normal/jump in a couple of windows, not a dramatic runaway spike. A modest, bounded shift is exactly what the real math predicts.

Suggested a concrete follow-up check: pull `p_trace` from the applied variant on real hardware (DIAG-format capture, not just the point-to-point HPC format currently in `seed1_applied_stf`) and check whether it settles into a sustained but flat elevated plateau during the attack window, versus climbing.

[USER]

How do we do something where there is more calculations for attack and it shows up distinctly

[ASSISTANT]

Recommended pivoting from "hope an existing computation's side effect correlates with attacks" to "deliberately gate genuinely extra computation behind the attack condition," so the instruction-count difference is intentional, not emergent.

Core idea: `compute_lambda_k()` (~680 multiply-adds, 10 divisions) currently runs unconditionally every step in `main_ekf_stf.c`, which is exactly why it doesn't move minstret. Proposed gating it behind `nis_alarm` instead:

```
if (nis_alarm) {
    lambda_k = compute_lambda_k(F, ekf.P, H, R_mat, FPft);
} else {
    lambda_k = 1.0;
}
```

Cited the known `nis_alarm` fire rates (normal ~2/299, drift ~51/299, jump ~147/299) as evidence this would produce a clean, large, non-noise-prone instruction-count difference, unlike the timing-based signals tried so far.

Flagged one honest limit: this inherits `nis_alarm`'s existing blind spots -- won't help with replay.

Offered to validate in Python first (extend `simulate_ekf_stf.py`) to quantify the expected minstret delta before touching firmware.

[USER]

[Selected lines 411-413 of `main_ekf_stf_applied.c`: the unconditional

lambda_k*FPFt+Q_mat scaling loop]

i want this thing to happen only if lambda_k is > 1 or greater than what we observe normally. It should not happen everytime that was my logic

[ASSISTANT]

Validated the logic: real hardware data showed lambda_k spiking as high as 23-26 during PURE NORMAL operation (from ekf_diag_normal.csv, decoupled main_ekf_stf.c), so gating on ">1.0" alone wouldn't be enough -- normal already exceeds 1 often.

Implemented a gated version in main_ekf_stf_applied.c:

```
double p_scale = (lambda_k > LAMBDA_APPLY_THRESHOLD) ? lambda_k : 1.0;
for (i = 0; i < EKF_N * EKF_N; i++)
    ekf.P[i] = p_scale * FPFt[i] + Q_mat[i];
```

Added LAMBDA_APPLY_THRESHOLD = 5.0 to ekf_config_stf.h as an explicit, flagged-as-unvalidated placeholder constant.

Along the way, discovered ekf_config_stf.h had been relocated to plots/seed1/ by an earlier reorganization commit, which would have broken compilation of both main_ekf_stf.c and main_ekf_stf_applied.c (both #include it expecting it at the top level). Asked the user how to handle this.

[USER]

Restore a top-level copy (Recommended)

[ASSISTANT]

Restored ekf_config_stf.h at the top level of adaptive_analysis_board_firmware/, with LAMBDA_APPLY_THRESHOLD added, leaving the plots/seed1/ copy as a historical record. Confirmed all four source files (ekf_config_stf.h, main_ekf_stf.c, main_ekf_stf_applied.c, tinyekf.h, trajectory.h) were present together again.

[USER]

in normal stf is lambda k applied at every step or is it applied only when there is a high value, because id ont understand the need of inflating covariance when lambda k is well withing the range say 1

[ASSISTANT]

Clarified: in main_ekf_stf.c (decoupled), lambda_k is NEVER applied to ekf.P, at any step, regardless of value -- ekf_predict() there is the plain unmodified library function. lambda_k is computed purely to be logged as a diagnostic.

main_ekf_stf_applied.c is the only file where it's ever applied, and (at that point in the conversation) was applying it every step unconditionally.

Addressed the conceptual point: when `lambda_k` is near its floor of 1.0, the scaling is already nearly a no-op mathematically -- that case was never really "wastefully inflating" anything. The actual problem the threshold fixes is the SPIKES (23-26 during normal, 5-8 during drift's own pre-attack period) -- those, if applied unconditionally, meaningfully perturb `P` even when nothing is wrong. That's what `LAMBDA_APPLY_THRESHOLD` targets.

[USER]

so how does the update happen can you explain?

[ASSISTANT]

Walked through `ekf_update()` from `tinyekf.h` step by step, tied to this project's concrete dimensions (`EKF_N=6`, `EKF_M=3`):

1. Innovation: $z_{hx} = z - hx$ (GPS reading minus accel-driven predicted position).
2. Innovation covariance: $S = H.P.H^T + R$ (total expected spread, same S used in `compute_nis()`).
3. Kalman gain: $G = P.H^T.S^{-1}$ -- computed via the Cholesky-based `invert()`, same division-heavy hot path discussed all session.
4. State update: $x = x + G.(z - hx)$.
5. Covariance update: $P = (I - G.H).P$ -- the "collapse" step, shrinking P proportional to how much the measurement actually contributed.

Summarized: measure the disagreement, weigh it by relative confidence, apply the weighted correction, then shrink uncertainty to reflect what was just learned.

[USER]

show me where `lambda k` is applied the applied stf file

[ASSISTANT]

Pointed to the exact lines (401-413 at the time) in `main_ekf_stf_applied.c`:

```
double FPft[EKF_N*EKF_N];
double lambda_k = compute_lambda_k(F, ekf.P, H, R_mat, FPft);
lambda_window_push(e_raw);
memcpy(ekf.x, fx, EKF_N * sizeof(double));
for (i = 0; i < EKF_N * EKF_N; i++)
    ekf.P[i] = lambda_k * FPft[i] + Q_mat[i];
int ok = ekf_update(&ekf, z_meas, hx, H, R_mat);
```

Explained the one line that actually does it (the loop writing directly into `ekf.P`), and noted this file skips the library's `ekf_predict()` entirely, reusing the `FPft` already computed inside `compute_lambda_k()` instead of letting `ekf_predict()` recompute an unscaled version.

[USER]

so when lambda is greater than 1, how does it bring P uncertainty down

[ASSISTANT]

Derived the scalar Kalman precision-addition identity:

$$1/p_{\text{post}} = 1/p + 1/r$$

$$p_{\text{post}} = p \cdot r / (p + r)$$

Showed that as $p \rightarrow \text{infinity}$ (what $\lambda_k > 1$ does), $p_{\text{post}} \rightarrow r$ from below, and can never exceed r . So no matter how large λ_k inflates the pre-update variance, the post-update variance is capped by the measurement noise r .

Explained why: inflating p doesn't ask the filter to "stay more uncertain forever" -- it asks it to trust the incoming measurement almost completely ($g \rightarrow 1$). Trusting a real measurement almost entirely leaves you with roughly that measurement's own noise level, not growing residual doubt.

Tied back to the actual code: R_{mat} here is $\text{SIGMA}_H^2 = 25 / \text{SIGMA}_V^2 = 100$, so λ_k inflating P pulls the posterior toward that ~ 25 - 100 range regardless of how extreme the pre-update spike was -- explaining why the "300x spike" from the file's docstring is a momentary, pre-update number, not what survives into p_{trace} .

[USER]

run the `simulate_ekf.py` and give me lambda k values for normal jump and drift run for seed1. also are there 300 lambda k values?

[ASSISTANT]

Ran `Trajectory/simulate_ekf_stf.py`'s `run_ekf_stf()` for seed1, all three modes. Results:

Mode	mean	min	max	std	values > 5.0
normal	1.2015	1.0000	4.2598	0.4472	0 / 299
jump	3.7609	1.0000	575.6482	33.4183	10 / 299
drift	1.2017	1.0000	4.2598	0.4475	0 / 299

Confirmed exactly 299 λ_k values per mode ($t=1$ through $t=299$, since $\text{TRAJ_LEN}-1=299$ and the loop starts at $t=1$, not $t=0$).

Flagged the important implication: drift's stats are essentially IDENTICAL to normal (same mean, same max to 4 decimals, same std) -- meaning `LAMBDA_APPLY_THRESHOLD=5.0` would never fire for drift at all (max 4.26 never crosses 5.0), making the gated P-scaling effectively a jump-only mechanism with that threshold, contributing nothing for drift specifically.

[USER]

how are there 300 lambda k values if its windowed 10?

[ASSISTANT]

Clarified: it's a SLIDING (rolling) window, not a batch/non-overlapping one. The window holds the most recent (up to) 10 innovations and updates by one at a time -- drop oldest, add newest, recompute. Every single step gets its own freshly computed lambda_k, using whichever 10 innovations are most recent as of that step. Window size controls how much smoothing goes into each value, not how many values you get in total.

Drew the analogy to a 10-day stock moving average: one new point plotted every day, not one point every 10 days. Same mechanic as ANIS (window_push()/window_mean(), WINDOW_SIZE=10) already in the firmware.

[USER]

you said lambda_k = 1 means filter has less uncertainty then why does lambda_k go till 4 for normal operation itself

[ASSISTANT]

Clarified the paraphrase: lambda_k=1 means "no extra inflation" (a relative baseline), not an absolute guarantee of low uncertainty. Two built-in reasons normal legitimately produces lambda_k up to 4 (or higher, per real hardware data):

Reason 1 -- structural bias: M_k deliberately excludes Q, but V_k naturally includes it. So even under perfect conditions, expected ratio = $1 + \text{trace}(\mathbf{H} \cdot \mathbf{Q} \cdot \mathbf{H}^T) / \text{trace}(\mathbf{M}_k)$ -- nudged above 1 by construction. Confirmed by data: normal's own mean lambda_k is 1.2015, not 1.0.

Reason 2 -- sampling noise: V_k is estimated from only 10 real samples, so it has genuine statistical variance window to window -- sometimes a noisier-than-typical stretch of ordinary GPS noise lands in the window, spiking ratio purely by chance.

Together: normal sits on an already-slightly-elevated baseline, with real random jitter on top -- occasional excursions to 4 (or into the 20s on real hardware) are expected from a perfectly healthy filter, not a sign of a problem. This is also why raw per-step lambda_k struggles to separate drift from normal -- drift's real signal has to clear an already-noisy baseline.

[USER]

using just anis window how many times did drift and jump fire?

[ASSISTANT]

Ran the baseline GPS+IMU filter (no lambda_k) for seed1, all three modes, counting anis_alarm fires (ANIS_THRESHOLD=4.377):

Mode	anis_alarm fires (of 299)
normal	15
jump	151
drift	119

Noted this is a much cleaner separation than anything `lambda_k` had produced -- drift 119 vs normal 15 is an unambiguous ~8x gap, unlike raw `lambda_k`'s near-identical drift/normal distributions.

[USER]

can you show me exactly when at what time points these fired?

[ASSISTANT]

Listed the exact t-values where `anis_alarm` fired, seed1:

Normal (15 fires): 62, 242-246, 264-272 (scattered isolated blips)

Jump (151 fires): 62, then continuous 150-299 (unbroken from attack onset to end of run, no recovery)

Drift (119 fires): 62, then continuous 160-275 (116 steps), then a gap, then isolated fires at 285, 286, then silence for the last 13 steps (287-299)

Highlighted two findings:

1. All three modes share the identical t=62 false alarm -- confirms normal/jump/drift share the same RNG-seeded GPS noise before `ATTACK_START=150`, since that data is literally identical across all three runs at that point.
 2. Jump's ANIS never recovers within the run; drift's tapers off near the end even though the absolute drift offset keeps growing the whole time (`DRIFT_MAX_M=200` never actually caps within this 300-step run, since $150+200=350$ exceeds `TRAJ_LEN`). Speculated this could reflect the filter's velocity state gradually, partially "leaning into" a sufficiently persistent drift late in the run, even with IMU resistance -- flagged as an open question for longer-duration drift attacks.
-

[USER]

so instead of checking if `lambda_k > 1` and inflating P can we use `anis` to decide whether to inflate P or not?

[ASSISTANT]

Agreed this is a better design than gating on raw `lambda_k` magnitude, given ANIS already cleanly separates drift/jump from normal while raw `lambda_k` does not.

Flagged the causality detail: at predict-time, THIS step's `nis/anis` don't exist yet (computed after update, from post-update P) -- so the gate must use last step's `anis` (`anis_prev`), consistent with the

existing one-step-lag principle already used for `lambda_k` itself.

Proposed:

```
double p_scale = (anis_prev > ANIS_THRESHOLD) ? lambda_k : 1.0;
```

Separating "is something wrong" (answered by ANIS) from "how much to inflate, given something's wrong" (still answered by `lambda_k`'s own ratio).

Flagged a risk worth validating: since ANIS stays elevated for 116+ consecutive steps during drift, this gate would stay open for a long stretch too, potentially reintroducing detection suppression in a different form. Offered to check this in `simulate_ekf_stf.py` first.

[USER]

no i also want an exclusive branching `ekf.P` is only inflated(branch taken) when `lambda_k` is greater than 1. if not nothing happens, i dont care if `lambda_k=1` so it doesnt matter. i want recomputation only when `anis>threshold`

[ASSISTANT]

Implemented the full two-level exclusive-branch design in `main_ekf_stf_applied.c`:

BRANCH 1 (gated on `anis_prev > ANIS_THRESHOLD`): `compute_lambda_k()` (~680 multiply-adds + 10 divisions) only runs when last step's ANIS already flagged something. Otherwise, calls the plain library `ekf_predict()` directly -- identical to `main_ekf_stf.c` for that step.

BRANCH 2 (nested inside branch 1): even when `compute_lambda_k()` DID run, `P` is only actually inflated if `lambda_k > 1.0` genuinely. If it landed exactly at the floor, reuses the already-computed `FPFt` for a plain unscaled predict instead of re-deriving it.

Code:

```
lambda_window_push(e_raw);    // stays unconditional -- cheap
double lambda_k = 1.0;
if (anis_prev > ANIS_THRESHOLD) {
    double FPFt[EKF_N*EKF_N];
    lambda_k = compute_lambda_k(F, ekf.P, H, R_mat, FPFt);
    memcpy(ekf.x, fx, EKF_N * sizeof(double));
    if (lambda_k > 1.0) {
        for (i = 0; i < EKF_N * EKF_N; i++)
            ekf.P[i] = lambda_k * FPFt[i] + Q_mat[i];
    } else {
        for (i = 0; i < EKF_N * EKF_N; i++)
            ekf.P[i] = FPFt[i] + Q_mat[i];
    }
} else {
    ekf_predict(&ekf, fx, F, Q_mat);
}
int ok = ekf_update(&ekf, z_meas, hx, H, R_mat);
```

Added static double `anis_prev = 0.0` as persistent state, updated at the end of each `ekf_step()` call (`anis_prev = anis;`) for the next call's branch decision.

Removed the now-superseded `LAMBDA_APPLY_THRESHOLD` from `ekf_config_stf.h`, since `ANIS_THRESHOLD` (already existing, already validated) replaced its role.

Rewrote the file's header docstring to accurately describe the new two-branch design, including an explicit "NOT YET VALIDATED" note: the old NIS/ANIS-degradation numbers (drift `nis_alarm` 51/299 -> 2/299) describe the PREVIOUS unconditional-application design, not this one -- the actual detection impact of this gated version still needs checking in `Trajectory/simulate_ekf_stf.py` before drawing conclusions.

[USER]

can you give me a text file with pur conversations after 5, i need to store it

[ASSISTANT]

Asked for clarification on what "after 5" meant (5pm, message #5, last 5 exchanges, or something else).

[USER]

After 5 PM today

[ASSISTANT]

Explained the limitation (no per-message timestamps available), anchored the range using the git push timestamp (e23c304 at 15:29:25) and the next file-edit timestamps (18:56-18:58 for the two-branch implementation), and produced this transcript.

END OF TRANSCRIPT